

Programming Assignment 1: Local Features and Object-Locked Video

Released Sep. 8
Target 8–10 hours

Due Sep. 29
Platform Google Colab

Points 100 + up to 10 EC
Submit Code + PDF + Hero Result

OVERVIEW

A photograph begins as an array of numbers. A useful vision system must transform those numbers into representations that preserve meaningful visual structure while becoming less sensitive to nuisance changes such as brightness, scale, rotation, blur, and viewpoint.

In this assignment, you will build a classical local-feature pipeline from the ground up. You will derive the mathematics behind Harris corner detection, implement the essential stages of feature detection and matching, and then use those features inside a larger video system.

Your final goal is to create an **object-locked smartphone video effect**: record a short handheld video containing a distinctive object, track that object across frames using local visual features, and stabilize the video so that the target appears approximately fixed while the surrounding scene moves.

The geometric alignment used in the final system is intentionally provided as a black box. You will derive and implement that geometry later in the course.

LEARNING OBJECTIVES

After completing this assignment, you should be able to:

- represent local image structure using gradients, vectors, and matrices;
- derive the Harris second-moment matrix from a local image-shift objective;
- interpret its eigenvalues geometrically as flat regions, edges, and corners;
- implement a local-feature detector and descriptor from basic image operations;
- explain why descriptors seek invariance to nuisance transformations;
- match local features across images and diagnose incorrect correspondences;
- connect classical feature computation to computational primitives used in modern learned vision systems;
- experimentally determine when a visual representation succeeds and when its assumptions break.

Part A — THINK: Local Image Structure

25 points | 1.5–2 hr

GenAI: NOT PERMITTED

Generative AI tools may not be used to generate, explain, solve, check, or rewrite answers for Part A. You may use course notes, textbooks, papers, conventional web resources, and course staff as allowed by the collaboration policy. All written work must be typeset in $\text{\LaTeX}$.

A1. From image shifts to the Harris matrix

10 pts

In lecture, we derived Harris from an unweighted shift error. Here you will analyze a photometrically compensated variant. Let $w_i \geq 0$, $\sum_i w_i = 1$, and define

$$E(\mathbf{d}, c) = \sum_i w_i [I(\mathbf{x}_i + \mathbf{d}) - I(\mathbf{x}_i) - c]^2, \quad \mathbf{d} = [u \ v]^\top,$$

where the unknown scalar c models a small additive brightness change between the two patches. Using $I(\mathbf{x}_i + \mathbf{d}) \approx I(\mathbf{x}_i) + \mathbf{g}_i^\top \mathbf{d}$:

1. derive the quadratic approximation $E(\mathbf{d}, c)$ in terms of the gradients $\mathbf{g}_i$;
2. minimize over c and show that $c^*(\mathbf{d}) = \bar{\mathbf{g}}^\top \mathbf{d}$, where $\bar{\mathbf{g}} = \sum_i w_i \mathbf{g}_i$;
3. show that the minimized error is $E(\mathbf{d}, c^*) \approx \mathbf{d}^\top M_c \mathbf{d}$ and derive $M_c = \sum_i w_i (\mathbf{g}_i - \bar{\mathbf{g}})(\mathbf{g}_i - \bar{\mathbf{g}})^\top$;
4. compare M_c with the ordinary Harris matrix $M = \sum_i w_i \mathbf{g}_i \mathbf{g}_i^\top$. What gradient structure is removed by compensation, and when might that help or hurt detection?

This is deliberately not the lecture derivation: the new step is eliminating a nuisance parameter and interpreting the resulting *centered* second-moment matrix.

A2. Work through one patch by hand

7 pts

The following 7×7 grayscale patch has row coordinate y increasing downward and column coordinate x increasing rightward:

$$I = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 3 & 3 & 3 & 3 \\ 0 & 0 & 1 & 3 & 5 & 5 & 5 \\ 0 & 0 & 1 & 3 & 5 & 7 & 7 \\ 0 & 0 & 1 & 3 & 5 & 7 & 9 \end{bmatrix}.$$

At the four marked locations $S = \{(3, 3), (4, 3), (3, 4), (4, 4)\}$ (coordinates are (x, y) , zero-indexed), compute gradients using the **central-difference operator** $I_x(x, y) = [I(x+1, y) - I(x-1, y)]/2$ and $I_y(x, y) = [I(x, y+1) - I(x, y-1)]/2$. Treat

$$M = \sum_{(x,y) \in S} \mathbf{g}(x, y) \mathbf{g}(x, y)^\top.$$

1. Show each finite-difference calculation and list the four gradient vectors.
2. Construct M explicitly.
3. Compute its two eigenvalues and corresponding eigenvector directions.
4. Classify the patch as most consistent with a flat region, edge, or corner.
5. Explain geometrically why this eigenvalue pattern corresponds to your answer.

Do not use a numerical linear-algebra package for this problem.

EIGENVECTOR HINT

For $M = \begin{bmatrix} a & b \\ b & d \end{bmatrix}$, first solve $\det(M - \lambda I) = 0$. For each eigenvalue, solve one nonzero row of $(M - \lambda I)\mathbf{v} = \mathbf{0}$, for example $(a - \lambda)v_x + bv_y = 0$, then normalize if desired. The sign of an eigenvector is arbitrary: $\mathbf{v}$ and $-\mathbf{v}$ describe the same direction.

A3. Predict detector behavior

4 pts

Without running code, predict how the Harris response should behave in each case below. Explain each prediction using gradients and/or the second-moment matrix.

- uniform-intensity patch;
- strong straight edge;
- corner;
- the same corner after adding a constant brightness offset to every pixel;
- the same corner after strong Gaussian blur.

A4. Representation and invariance

4 pts

Consider three possible descriptors for a local patch: (1) raw pixel intensities, (2) image gradients, and (3) spatial histograms of gradient orientation. For each representation, identify one nuisance transformation to which it should be more robust than the previous representation and one piece of information that may be lost.

CLASSICAL → MODERN CONNECTION

Harris is hand-designed, but many of its building blocks reappear in modern learned vision systems. Image derivatives are small convolutional filters; their responses are combined through products and local aggregation; later operations select useful structure from those representations. CNNs learn many such filters rather than fixing them in advance, while attention-based architectures also explicitly compute interactions among feature representations. The mathematics is not identical, but filtering, feature interaction, aggregation, normalization, and nonlinear selection are recurring computational primitives. Look for these operations again when we study CNNs and Transformers.

Part B — BUILD: Detect, Describe, Match

35 points | about 3 hr

GenAI: NOT PERMITTED FOR REQUIRED IMPLEMENTATION

You may use NumPy, PyTorch, and OpenCV documentation together with course materials, but generative AI may not generate the required TODO implementations. The starter code provides image loading, plotting, test images, video utilities, and geometric alignment infrastructure that is not central to the learning objectives. Do not call OpenCV feature detectors, descriptors, or matchers in the graded functions.

B1. Read the pipeline

5 pts

Before implementing anything, inspect the starter code and answer these five code-reading questions in the written PDF. They also appear, identically, in the starter notebook.

1. A grayscale image has shape (480, 640). What are the shapes of I_x and I_y , and at which array entry is keypoint [137, 82] accessed?
2. Inspect the supplied separable Gaussian aggregation helper. Why does its output preserve (H, W) , and how are samples beyond the image boundary defined?
3. With `patch_size=16`, `cells=4`, and `bins=8`, state the exact patch bounds around integer keypoint (x, y) , the spatial size of each cell, and the descriptor length.
4. What does L2 normalization do to the descriptor under a global positive contrast scaling of the image? What descriptor is produced by a flat patch?
5. The matcher receives reference descriptors as its first argument and current-frame descriptors as its second. Given `matches`, which reference/current keypoint arrays are passed to the supplied estimator, and in which direction does the estimated transform map points?

B2. Harris detector

12 pts

Implement the essential Harris pipeline in the provided TODO functions:

- `compute_image_gradients(image) -> (Ix, Iy)`: input a float grayscale array (H, W) ; return two float arrays (H, W) in the supplied derivative convention.
- `compute_second_moment(Ix, Iy, sigma) -> (Sxx, Sxy, Syy)`: input aligned gradient arrays and a positive window scale; return Gaussian-aggregated product arrays (H, W) .
- `compute_corner_response(Sxx, Sxy, Syy, k) -> R`: input the three moment components and Harris constant k ; return $R = \det(M) - k \operatorname{tr}(M)^2$ as (H, W) .
- `nonmax_suppression(R, threshold, radius, max_points) -> (keypoints, scores)`: input a response map and selection parameters; return keypoints as an $(N, 2)$ array of (x, y) coordinates and their $(N,)$ scores, sorted high to low, with $N \leq \text{max_points}$.

Your notebook must visualize I_x, I_y , the corner-response image, and the final selected interest points overlaid on the input image. The course provides routine Gaussian/windowing infrastructure. The released defaults are $\sigma = 1.5$ and $k = 0.04$, and the shared random seed is 16720.

NONMAXIMUM SUPPRESSION (NMS)

Thresholding alone often returns a dense blob of nearby pixels around one physical corner. NMS converts the response map into a sparse, repeatable set of interest points. The released method is **deterministic greedy disk suppression**: discard nonfinite responses, pixels within `radius` of the image border, and responses below `threshold` times the largest positive finite response; process the survivors by decreasing score, breaking ties by increasing y and then increasing x ; accepting a point suppresses every remaining candidate at Euclidean distance $\leq \text{radius}$; stop after `max_points`. The public tests check these rules exactly, including tie order, the exact-distance boundary, and empty-output shapes.

B3. A compact gradient descriptor

10 pts

Implement a simplified local descriptor around each detected feature. The goal is *not* to reproduce every detail of SIFT. Your descriptor should:

1. extract a local window around a keypoint;
2. compute gradient magnitude and orientation;
3. divide the patch into spatial cells;
4. accumulate a small orientation histogram in each cell;
5. normalize the final descriptor vector.

The released settings are a 16×16 patch, a 4×4 cell grid, and 8 orientation bins, so $D = 4^2 \cdot 8 = 128$. Patches beyond the image border are zero padded; there is no dominant-orientation alignment, interpolation, clipping, or Gaussian weighting; normalize each descriptor as $\mathbf{d}/(\|\mathbf{d}\|_2 + 10^{-8})$. The required function interface is `describe_keypoints(Ix, Iy, keypoints, patch_size, cells, bins) -> descriptors`: inputs are gradient arrays (H, W) , keypoints $(N, 2)$ in (x, y) order, and descriptor settings; output is a float array (N, D) where $D = \text{cells}^2 \text{bins}$.

In addition to reporting descriptor vectors, visualize what they capture for at least two keypoints:

- the image patch with its cell grid and keypoint center;
- a gradient-magnitude image with a sparse orientation “quiver” overlay;
- the orientation histogram for every spatial cell (a small-multiple or glyph visualization);

- a side-by-side comparison of descriptors for the original patch and one perturbed version.

Annotate which orientations/cells dominate and what changed after the perturbation.

B4. Feature matching

8 pts

Implement descriptor matching with a **Euclidean nearest-neighbor ratio test**, the released reliability rule, using default ratio 0.80: accept a source descriptor only when $d_1/d_2 < \text{ratio}$ over the valid destination candidates, with confidence $1 - d_1/(d_2 + 10^{-8})$, and return matches sorted by decreasing confidence and then by the two original indices. Reject nonfinite and near-zero descriptor rows; fewer than two valid destination descriptors produce no matches. Produce a visualization showing correspondences between two supplied images and briefly identify one correct and one incorrect match. The required interface is `match_descriptors(desc1, desc2, ratio) -> (matches, confidence)`: input descriptor matrices (N_1, D) and (N_2, D) plus the released threshold; return integer index pairs $(K, 2)$ and confidence scores $(K,)$ sorted from most to least reliable. Empty inputs must return correctly shaped empty arrays.

Part C — UNDERSTAND: Stress-Test Your Representation

15 points | 1.5–2 hr

GenAI: PERMITTED, WITH DISCLOSURE

Making the video work is not enough. Determine when and why your representation succeeds or fails. Generative AI may help organize experiments or plots, but your conclusions must be supported by your own recorded evidence, and every permitted use must be disclosed in your submission.

C1. Real-world invariance experiments

8 pts

Run exactly these four conditions against **one shared reference view that you capture yourself**:

1. a real illumination change, re-photographed with your camera;
2. a real physical scale and/or viewpoint change, re-photographed with your camera;
3. synthetic Gaussian blur; and
4. synthetic in-plane rotation,

with the two synthetic conditions applied to your reference photo using the supplied transforms so their parameters are reproducible. Use at least one clearly nontrivial severity for each condition and record the physical setup or synthetic parameter.

For every condition, report the same five quantities: number of reference features, number of destination features, number of ratio-test-accepted matches, number of robust-estimator inliers, and inlier fraction (defined as inliers over accepted matches, reported as zero when no match is accepted). Put all four rows in one compact table, show a readable match/inlier visualization for each condition, and give a one- or two-sentence result summary per row.

C2. Connect experiment to theory

4 pts

Choose two conditions from C1. For each, state what you expected before running the experiment, describe what actually happened, and explain the result using an idea from Part A or Part B. At least one analysis must involve descriptor invariance. Label prediction and observation separately so hindsight does not replace a prediction.

C3. Failure case

3 pts

Find one case where your tracker clearly fails. Show the failure and diagnose the most likely cause with pipeline evidence. Examples include insufficient texture, repetitive patterns, severe blur, large viewpoint change, occlusion, too few detected points, or incorrect descriptor matches. The purpose is diagnosis, not fixing every possible failure.

Part D — CREATE: Object-Locked Video

25 points | 2.5–3 hr

GenAI: PERMITTED AND ENCOURAGED, WITH DISCLOSURE

You may use generative AI for coding, debugging, refactoring, video processing, visualization, and presentation. It may not create or repair the graded Part B detector, descriptor, or matcher. You remain responsible for understanding and testing everything you submit, and for disclosing every permitted use.

D1. Capture your video

4 pts

Using a smartphone, record a 10–20 second handheld video containing a visually distinctive target such as a poster, book cover, sign, piece of artwork, package, logo, or other textured approximately rigid object. Avoid featureless, glossy, or highly repetitive surfaces. Your clip should contain some combination of camera translation, rotation, scale change, and moderate viewpoint change. Record in landscape at up to 720p and 30 fps; the target should occupy roughly 25–60% of the frame and stay visible. Do not record non-consenting people, private documents, or personal information on screens. A course-provided capture alternative is available only with advance staff approval for access, privacy, safety, or accessibility

reasons.

D2. Track and stabilize

8 pts

Use your detector, descriptor, and matcher from Part B to provide correspondences between video frames. The course provides infrastructure for video decoding, frame sampling, robust geometric estimation, frame warping, smoothing/stabilization, and video encoding: the supplied harness matches your first-frame ROI reference descriptors to every current frame, estimates current-to-reference motion robustly, holds a failed transform for at most five frames, and then visibly flags frames until tracking resumes. Your goal is a video in which the chosen target remains approximately fixed while the surrounding scene moves.

GEOMETRY BLACK BOX

The transformation used to align frames is intentionally provided in this assignment. Later in the course you will derive and implement the geometry behind this operation yourself. For now, your responsibility is to produce reliable local-feature correspondences and understand when they succeed or fail.

D3. Hero Result

10 pts

Produce one polished 10–20 second final video demonstrating your system. The result should clearly show the object-locked effect, be understandable without opening your notebook, use your own captured video, and be something you would be comfortable showing outside the class. Deliver a labeled original/object-locked comparison encoded as H.264/yuv420p MP4, at most 720p and 30 fps, and at most 50 MB. Optional presentation elements such as a short title, caption, music, or simple editing are allowed but do not replace the technical requirements.

D4. Comprehension reflection

3 pts

In approximately 150–250 words, follow one of two equally valid paths. **If you used generative AI** in the permitted parts: (1) what did it help you implement; (2) how does the most important AI-assisted portion of your code work; (3) what did you change, debug, or redesign from the initially generated version? **If you used no generative AI**: state that explicitly, explain your most important independently implemented component at the same technical depth, and describe a concrete debugging or validation step you performed.

Part E — EXPLORE: Optional Extension

up to +10 EC | about 1 hr

GenAI: PERMITTED, WITH DISCLOSURE

Do something we did not explicitly ask you to do. Extend the method, improve it, break it, compare it, or apply it somewhere unexpected. Examples include recovering after occlusion, tracking multiple targets, comparing against OpenCV SIFT or a modern learned matcher, improving stabilization, designing a more invariant descriptor, or deliberately constructing an adversarial scene. Extension and showcase credit are jointly capped at 10 extra-credit points.

- +5 Meaningful additional exploration with a clear result and short explanation.
- +10 Particularly thoughtful, technically interesting, creative, or ambitious exploration.

OPTIONAL COURSE SHOWCASE

Every student must create a Hero Result, but public participation is optional. Students who want to participate may post their final artifact to the course Slack #showcase channel by the announced deadline.

Showcase participation: +1 extra-credit point. This point is for participating, not for receiving votes. After submissions close, voting will open for a fixed period using one designated Slack reaction. Selected videos may be shown briefly in class, and the class-voted winner may receive a small prize or recognition. Voting does not determine academic credit beyond the +1 participation point, and all extra credit remains within the 10-point cap.

SUBMISSION

Submit the following through the course submission system:

- your completed `pal.py` and the executed starter notebook with the required evidence outputs retained;
- one $\LaTeX$ -generated PDF containing all written responses, every required figure/table, and the D4 reflection;
- the final Hero Result video (H.264/yuv420p MP4, at most 720p and 30 fps, 10–20 seconds, at most 50 MB);
- acknowledgment of collaborators, outside resources, and AI assistance as required by the course policy.

All required code must execute in Google Colab on readily available course hardware. Keep your raw captures locally until grades are final.

LATEX + GRADESCOPE FORMAT

Use the provided answer-template `.tex` file; do not retype the questions or change its problem labels. Compile and upload **one PDF** containing all written answers. Begin each labeled subproblem (A1, A2, and so on) on a new page using the page breaks already included in the template; an answer may continue onto additional pages when needed. After upload, assign every PDF page containing an answer to that question in Gradescope and verify the rendered preview. Gradescope permits several questions on one page and several pages for one question, so one question per page is not required for correctness. It is recommended here because Gradescope's "Assign Pages Sequentially" workflow is fastest when questions begin on separate, ordered pages and it reduces missed page tags. Do not insert unrelated cover or scratch pages.